FINAL PROJECT 1
(SystemVerilog)
FIFO

Name: Ahmed goda sharawy

VERIFICATION PLANE

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
FIFO_1	When the reset is asserted (rst_n = 0), the internal write pointer, read pointer, and count should reset to 0 (the FIFO is considered empty).	Directed at the start of the sim, then randomized with a constraint that drives rst_n to be high (not in reset) most of the simulation time (95%).	-	Concurrent assertion to check wr_ptr, rd_ptr, and count reset to 0.
FIFO_2	The reset should take effect immediately, without waiting for a clock edge (asynchronous behavior).	Directed reset assertion at the start of the simulation before randomization begins.	-	Immediate assertion (in the top file) to check the asynchronous reset functionality.
FIFO_3	When wr_en is active and the FIFO is not full, wr_ack should be asserted to confirm the write operation.	Randomization for wr_en with distribution WR_EN_ON_DIST to be 70% of the time.	Cross coverage between wr_en, rd_en, and wr_ack.	Concurrent assertion to check for the wr_ack.
FIFO_4	If a write is attempted when the FIFO is full, overflow should be asserted.	Randomization for wr_en (WR_EN_ON_DIST) while repeated writes drive the FIFO to the full state.	Cross coverage between wr_en, rd_en, and overflow.	Concurrent assertion to check for the overflow.
FIFO_5	If a read is attempted when the FIFO is empty, underflow should be asserted.	Randomization for rd_en (RD_EN_ON_DIST) while the FIFO is empty.	Cross coverage between wr_en, rd_en, and underflow.	Concurrent assertion to check for the underflow.
FIFO_6	When the internal count is zero, the empty flag should be asserted.	-	Cross coverage between wr_en, rd_en, and empty.	Concurrent assertion to check for the empty flag.
FIFO_7	When the internal count equals FIFO_DEPTH, the full flag should be asserted.	-	Cross coverage between wr_en, rd_en, and full.	Concurrent assertion to check for the full flag.
FIFO_8	When the count reaches FIFO_DEPTH - 1, almostfull should be asserted.	-	Cross coverage between wr_en, rd_en, and almostfull.	Concurrent assertion to check for the almostfull flag.
FIFO_9	When the count equals 1, the almostempty signal should be asserted.	-	Cross coverage between wr_en, rd_en, and almostempty.	Concurrent assertion to check for the almostempty flag.
FIFO_10	After writing FIFO_DEPTH entries, the write pointer should wrap back around to 0.	Repeated/randomized writes (wr_en high, WR_EN_ON_DIST) to fill the FIFO past its depth multiple times.	-	Concurrent assertion to check for the wr_ptr wraparound.
FIFO_11	After reading FIFO_DEPTH entries, the read pointer should wrap back around to 0.	Repeated/randomized reads (rd_en high, RD_EN_ON_DIST) to empty the FIFO past its depth multiple times.	-	Concurrent assertion to check for the rd_ptr wraparound.
FIFO_12	The write pointer cannot exceed the FIFO_DEPTH entries at any given time.	-	-	Concurrent assertion to check for the wr_ptr threshold.
FIFO_13	The read pointer cannot exceed the FIFO_DEPTH entries at any given time.	-	-	Concurrent assertion to check for the rd_ptr threshold.
FIFO_14	The internal counter cannot exceed the FIFO_DEPTH entries at any given time.	-	-	Concurrent assertion to check for the count threshold.
FIFO_15	When a read is performed, data_out should return the correct value in FIFO (first-in first-out) order.	Randomization for data_in, wr_en (WR_EN_ON_DIST), and rd_en (RD_EN_ON_DIST) per the FIFO_transaction	-	Golden/reference model only (scoreboard check_data / reference_model).
FIFO_16	When both wr_en and rd_en are asserted while the FIFO is empty, only the write should take place (no read).	Natural overlap of the independently-randomized wr_en/rd_en distributions while the FIFO is empty (e.g. right after reset).	Covered by the wr_en-rd_en-empty cross (bin: wr_en=1, rd_en=1, empty=1).	Golden/reference model (reference_model must model this boundary case correctly).
FIFO_17	When both wr_en and rd_en are asserted while the FIFO is full, only the read should take place (no write).	Natural overlap of the independently-randomized wr_en/rd_en distributions while the FIFO is full.	Covered by the wr_en-rd_en-full cross (bin: wr_en=1, rd_en=1, full=1).	Golden/reference model (reference_model must model this boundary case correctly).

RTL

codes > FIFO.sv > FIFO

```
////////////////////////////////////////////////////////////////////////////////
// Author: Kareem Waseem
// Course: Digital Verification using SV & UVM
//
// Description: FIFO Design
//
////////////////////////////////////////////////////////////////////////////////
module FIFO(FIFO_interface.DUT F_if);


localparam max_fifo_addr = $clog2(F_if.FIFO_DEPTH);

reg [F_if.FIFO_WIDTH-1:0] mem [F_if.FIFO_DEPTH-1:0];

reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
reg [max_fifo_addr:0] count;

always @(posedge F_if.clk or negedge F_if.rst_n) begin
    if (!F_if.rst_n) begin
        wr_ptr <= 0;
    end
    else if (F_if.wr_en && count < F_if.FIFO_DEPTH) begin
        mem[wr_ptr] <= F_if.data_in;
        F_if.wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;
    end
    else begin
        F_if.wr_ack <= 0;
        if (F_if.full & F_if.wr_en)
            F_if.overflow <= 1;
        else
            F_if.overflow <= 0;
    end
end
```

```
always @(posedge F_if.clk or negedge F_if.rst_n) begin
    if (!F_if.rst_n) begin
        rd_ptr <= 0;
        F_if.data_out <= '0;   // bug 1 : reset data out to zero
    end
    else if (F_if.rd_en && count != 0) begin
        F_if.data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
    end
end

always @(posedge F_if.clk or negedge F_if.rst_n) begin
    if (!F_if.rst_n) begin
        count <= 0;
    end
    else begin
        if  ( ({F_if.wr_en, F_if.rd_en} == 2'b10) && !F_if.full)
            count <= count + 1;
        else if ( ({F_if.wr_en, F_if.rd_en} == 2'b01) && !F_if.empty)
            count <= count - 1;
            else if ( {F_if.wr_en, F_if.rd_en} == 2'b11) begin      // bug 2: {when write en and read en} = 2'b11
            if (F_if.empty)
                count <= count + 1;
            else if (F_if.full)
                count <= count - 1;
            end
    end
end

assign F_if.full = (count == F_if.FIFO_DEPTH)? 1 : 0;
assign F_if.empty = (count == 0)? 1 : 0;
assign F_if.underflow = (F_if.empty && F_if.rd_en)? 1 : 0;
assign F_if.almostfull = (count == F_if.FIFO_DEPTH-2)? 1 : 0;
assign F_if.almostempty = (count == 1)? 1 : 0;
```

```
72  //====================== ASSERTIONS ======================
73  `ifdef SIM
74
75  // a.Reset Behavior
76  property p_reset;
77      @(posedge F_if.clk) !F_if.rst_n |=> (!wr_ptr && !rd_ptr && !count);
78  endproperty
79  assert property (p_reset);
80  cover  property (p_reset);
81
82  // b.Write Acknowledge (wr_ack)
83  property p_wr_ack;
84      @(posedge F_if.clk) disable iff (!F_if.rst_n)
85      (F_if.wr_en && !F_if.full) |=> F_if.wr_ack;
86  endproperty
87  assert property (p_wr_ack);
88  cover  property (p_wr_ack);
89
90  // c.Overflow Detection
91  property p_overflow;
92      @(posedge F_if.clk) disable iff (!F_if.rst_n)
93      (F_if.wr_en && F_if.full) |=> F_if.overflow;
94  endproperty
95  assert property (p_overflow);
96  cover  property (p_overflow);
97
98  // d.Underflow Detection
99  property p_underflow;
100     @(posedge F_if.clk) disable iff (!F_if.rst_n)
101     (F_if.rd_en && F_if.empty) |-> F_if.underflow;
102 endproperty
103 assert property (p_underflow);
104 cover  property (p_underflow);
105
106 // e.Empty Flag Assertion
107 property p_empty;
108     @(posedge F_if.clk) disable iff (!F_if.rst_n)
109     (!count ) |-> F_if.empty;
110 endproperty
111 assert property (p_empty);
112 cover  property (p_empty);
113
114 // f.Full Flag Assertion
115 property p_full;
116     @(posedge F_if.clk) disable iff (!F_if.rst_n)
117     (count == F_if.FIFO_DEPTH ) |-> F_if.full;
118 endproperty
119 assert property (p_full);
120 cover  property (p_full);
121
122 // g.Almost Full Condition
123 property p_almostfull;
124     @(posedge F_if.clk) disable iff (!F_if.rst_n)
125     (count == F_if.FIFO_DEPTH - 2 ) |-> F_if.almostfull;
126 endproperty
127 assert property (p_almostfull);
128 cover  property (p_almostfull);
129
130 // h.Almost Empty Condition
131 property p_almostempty;
132     @(posedge F_if.clk) disable iff (!F_if.rst_n)
133     (count == 1'b1 ) |-> F_if.almostempty;
134 endproperty
135 assert property (p_almostempty);
136 cover  property (p_almostempty);
137
138 // i.Pointer Wraparound
139 // 1. write pointer
140 property p_wraparound_wr_ptr;
141     @(posedge F_if.clk) disable iff (!F_if.rst_n)
142     (wr_ptr == F_if.FIFO_DEPTH - 1'b1  ) && F_if.wr_en && !F_if.full |=> (wr_ptr == 0);
143 endproperty
144 assert property (p_wraparound_wr_ptr);
145 cover  property (p_wraparound_wr_ptr);
146
```

```
147   // 2.read pointer
148   property p_wraparound_rd_ptr;
149       @(posedge F_if.clk) disable iff (!F_if.rst_n)
150       (rd_ptr == F_if.FIFO_DEPTH - 1'b1  ) && F_if.rd_en && !F_if.empty |=> (rd_ptr == 0);
151   endproperty
152   assert property (p_wraparound_rd_ptr);
153   cover  property (p_wraparound_rd_ptr);
154
155   // j. Pointer threshold
156   // 1. write pointer
157   property p_threshold_wr_ptr;
158       @(posedge F_if.clk) disable iff (!F_if.rst_n)
159       (wr_ptr <= F_if.FIFO_DEPTH - 1'b1);
160   endproperty
161   assert property (p_threshold_wr_ptr);
162   cover  property (p_threshold_wr_ptr);
163
164   // 2.read pointer
165   property p_threshold_rd_ptr;
166       @(posedge F_if.clk) disable iff (!F_if.rst_n)
167       (rd_ptr <= F_if.FIFO_DEPTH - 1'b1);
168   endproperty
169   assert property (p_threshold_rd_ptr);
170   cover  property (p_threshold_rd_ptr);
171
172   // 3.count
173   property p_threshold_count;
174       @(posedge F_if.clk) disable iff (!F_if.rst_n)
175       (count <= F_if.FIFO_DEPTH);
176   endproperty
177   assert property (p_threshold_count);
178   cover  property (p_threshold_count);
```

```
180   // 7. ensuring that asynchronous reset is working as expected
181   // v. Asynchronous Reset
182   always_comb begin
183       if (!F_if.rst_n) begin
184           assert final (wr_ptr == 0);
185           assert final (rd_ptr == 0);
186           assert final (count == 0);
187           cover (wr_ptr == 0 && rd_ptr == 0 && count == 0);
188       end
189   end
190   `endif
191
192   endmodule
```

Shared_pkg

codes > shared_pkg.sv > { } shared_pkg

```
1   package shared_pkg;
2
3       bit     test_finished = 0;
4       integer correct_count = 0;
5       integer error_count   = 0;
6       event   drv_done; // event
7
8   endpackage
```

Transaction_pkg

codes > FIFO_transaction.sv > { } FIFO_transaction > FIFO_transaction

```
1   package FIFO_transaction;
2   parameter FIFO_WIDTH = 16;
3   class FIFO_transaction;
4
5   // signals
6   bit clk;
7   rand logic [FIFO_WIDTH-1:0] data_in;
8   rand logic rst_n, wr_en, rd_en;
9   logic  [FIFO_WIDTH-1:0] data_out;
10  logic  wr_ack, overflow;
11  logic  full, empty, almostfull, almostempty, underflow;
12  integer RD_EN_ON_DIST;
13  integer WR_EN_ON_DIST;
14
15  // constructor
16  function new(integer RD_EN_ON_DIST = 30 ,integer WR_EN_ON_DIST= 70);
17  this.WR_EN_ON_DIST =WR_EN_ON_DIST;
18  this.RD_EN_ON_DIST = RD_EN_ON_DIST;
19
20  endfunction
21
22  // consructor
23  constraint reset_c {rst_n dist {1:=95 , 0 :=5};}
24  constraint write_c {wr_en dist {1:=WR_EN_ON_DIST , 0:=100 - WR_EN_ON_DIST};}
25  constraint read_c {rd_en dist  {1:=RD_EN_ON_DIST , 0:=100 - RD_EN_ON_DIST};}
26
27
28
29
30  endclass
31
32  endpackage
```

Coverage_pkg

codes > FIFO_coverage.sv > { } FIFO_coverage > FIFO_coverage

```
1   package FIFO_coverage;
2   import FIFO_transaction::*;
3   class FIFO_coverage;
4
5   // object FIFO_transaction
6   FIFO_transaction F_cvg_txn;
7
8
9   covergroup CovCode;
10
11  // ============= COVERPOINTS INPUTS =================
12
13  // coverpoint write
14  wr_en_cp : coverpoint F_cvg_txn.wr_en {
15      bins wr_en_0 = {0};
16      bins wr_en_1 = {1};
17  }
18
19  //  coverpoint read
20  rd_en_cp : coverpoint F_cvg_txn.rd_en {
21      bins rd_en_0 = {0};
22      bins rd_en_1 = {1};
23  }
24
25  // ============= COVERPOINTS OUTPUTS =================
26
27  // coverpoint Write Acknowledge
28  wr_ack_cp : coverpoint F_cvg_txn.wr_ack {
29      bins wr_ack_0 = {0};
30      bins wr_ack_1 = {1};
31  }
32
33  // coverpoint overflow
34  overflow_cp : coverpoint F_cvg_txn.overflow {
35      bins overflow_0 = {0};
36      bins overflow_1 = {1};
37  }
38
39  // coverpoint full
40  full_cp : coverpoint F_cvg_txn.full {
41      bins full_0 = {0};
42      bins full_1 = {1};
43  }
44
```

```
39   // coverpoint full
40   full_cp : coverpoint F_cvg_txn.full {
41       bins full_0 = {0};
42       bins full_1 = {1};
43   }
44
45   // coverpoint empty
46   empty_cp : coverpoint F_cvg_txn.empty {
47       bins empty_0 = {0};
48       bins empty_1 = {1};
49   }
50
51   // coverpoint almostfull
52   almostfull_cp : coverpoint F_cvg_txn.almostfull {
53       bins almostfull_0 = {0};
54       bins almostfull_1 = {1};
55   }
56
57   // coverpoint almostempty
58   almostempty_cp : coverpoint F_cvg_txn.almostempty {
59       bins almostempty_0 = {0};
60       bins almostempty_1 = {1};
61   }
62
63   // coverpoint underflow
64   underflow_cp : coverpoint F_cvg_txn.underflow {
65       bins underflow_0 = {0};
66       bins underflow_1 = {1};
67   }
68
69
```

```
70   // ================== CROSS COVERAGE ====================
71
72   cross_1 : cross wr_en_cp , rd_en_cp , wr_ack_cp;
73   cross_2 : cross wr_en_cp , rd_en_cp , overflow_cp;
74   cross_3 : cross wr_en_cp , rd_en_cp , full_cp;
75   cross_4 : cross wr_en_cp , rd_en_cp , empty_cp;
76   cross_5 : cross wr_en_cp , rd_en_cp , almostfull_cp;
77   cross_6 : cross wr_en_cp , rd_en_cp , almostempty_cp;
78   cross_7 : cross wr_en_cp , rd_en_cp , underflow_cp{
79       illegal_bins impossible = binsof (rd_en_cp) intersect {0} && binsof (underflow_cp) intersect {1};
80   }
81
82   endgroup
83
84   // e. Create a void function inside it named sample_data that takes one input named F_txn. This
85   //input is an object of class FIFO_transaction. This function will do the following
86   function void sample_data(FIFO_transaction F_txn);
87   F_cvg_txn = F_txn;
88   CovCode.sample();
89   endfunction
90   function new();
91   CovCode = new();
92   endfunction
93
94   endclass
95
96   endpackage
```

Scoreboard_pkg

codes > FIFO_scoreboard.sv > { } FIFO_scoreboard > FIFO_scoreboard

```
1   package FIFO_scoreboard;
2   import FIFO_transaction::*;
3   import shared_pkg::*;
4   parameter FIFO_DEPTH = 8;
5   class FIFO_scoreboard;
6
7   //  reference outputs
8
9   logic  [FIFO_WIDTH-1:0] data_out_ref ='0;
10
11
12  // QUEUE -----> SMALL FIFO TO CHECK DATA OUT
13  reg [FIFO_WIDTH - 1:0] queue[$];
14
15  // reference model
16  task reference_model(FIFO_transaction F_txn);
17      bit was_empty, was_full;
18
19      if (!F_txn.rst_n) begin
20          queue.delete();
21          data_out_ref = '0;
22      end
23      else begin
24          was_empty = (queue.size() == 0);
25          was_full  = (queue.size() == FIFO_DEPTH);
26
27          // write
28          if (F_txn.wr_en && !was_full)
29              queue.push_back(F_txn.data_in);
30
31          // read
32          if (F_txn.rd_en && !was_empty)
33              data_out_ref = queue.pop_front();
34      end
35  endtask
36
```

```
37  // check out
38  task check_data(FIFO_transaction F_txn);
39
40          // call task reference model
41          reference_model(F_txn);
42      if (F_txn.data_out !== data_out_ref)begin
43          $display("ERROR | refrence =%h , actual =%h", data_out_ref,F_txn.data_out);
44          error_count++;
45      end
46      else begin
47          $display("PASS | refrence =%h , actual =%h", data_out_ref,F_txn.data_out);
48          correct_count++;
49      end
50
51  endtask
52  endclass
53  endpackage
```

Monitor

codes > FIFO_monitor.sv > FIFO_monitor

```
module FIFO_monitor(FIFO_interface.MONITOR F_if);

import FIFO_transaction::*;
import FIFO_coverage::*;
import FIFO_scoreboard::*;
import shared_pkg::*;

// handls
FIFO_transaction F_txn;
FIFO_coverage    F_cvg;
FIFO_scoreboard  F_sb;

initial begin

    // objects
    F_txn = new();
    F_cvg = new();
    F_sb  = new();

    forever begin
    @(drv_done); // event

    // inputs
    F_txn.rst_n   = F_if.rst_n;
    F_txn.wr_en   = F_if.wr_en;
    F_txn.rd_en   = F_if.rd_en;
    F_txn.data_in = F_if.data_in;

    // outputs
    F_txn.data_out    = F_if.data_out;
    F_txn.wr_ack      = F_if.wr_ack;
    F_txn.overflow    = F_if.overflow;
    F_txn.full        = F_if.full;
    F_txn.empty       = F_if.empty;
    F_txn.almostfull  = F_if.almostfull;
    F_txn.almostempty = F_if.almostempty;
    F_txn.underflow   = F_if.underflow;

    fork
    // send coverage
    F_cvg.sample_data(F_txn);
    // send scoreboard
    F_sb.check_data(F_txn);
    join


end
end

initial begin
    wait(test_finished);

    $display("========================================");
    $display("Simulation Finished");
    $display("Correct Count = %0d", correct_count);
    $display("Error Count   = %0d", error_count);
    $display("========================================");

    $finish;
end
endmodule
```

Inerface

codes > FIFO_interface.sv > FIFO_interface > MONITOR

```
interface FIFO_interface (clk);
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;
input bit clk;
logic  [FIFO_WIDTH-1:0] data_in;
logic  rst_n, wr_en, rd_en;
logic  [FIFO_WIDTH-1:0] data_out;
logic  wr_ack, overflow;
logic full, empty, almostfull, almostempty, underflow;

modport DUT (input data_in, clk, rst_n, wr_en, rd_en, output data_out, wr_ack, overflow, full, empty, almostfull, almostempty, underflow);
modport TEST (output data_in, rst_n, wr_en, rd_en, input  clk,data_out, wr_ack, overflow, full, empty, almostfull, almostempty, underflow);
modport MONITOR (input data_in, clk, rst_n, wr_en, rd_en,data_out, wr_ack, overflow, full, empty, almostfull, almostempty, underflow);

endinterface
```

TestBench

codes > FIFO_tb.sv > FIFO_tb

```
module FIFO_tb(FIFO_interface.TEST F_if);
import FIFO_transaction::*;
import shared_pkg::*;
FIFO_transaction F_txn;

// Stimulus Generation
initial begin
    F_txn = new();
    assert_reset();

    repeat(20000) begin
        @(negedge F_if.clk);
        ->drv_done;  // go event
        #0;
       assert(F_txn.randomize());
       F_if.rst_n = F_txn.rst_n;
       F_if.data_in = F_txn.data_in;
       F_if.wr_en = F_txn.wr_en;
       F_if.rd_en = F_txn.rd_en;
    end

    test_finished = 1;

end

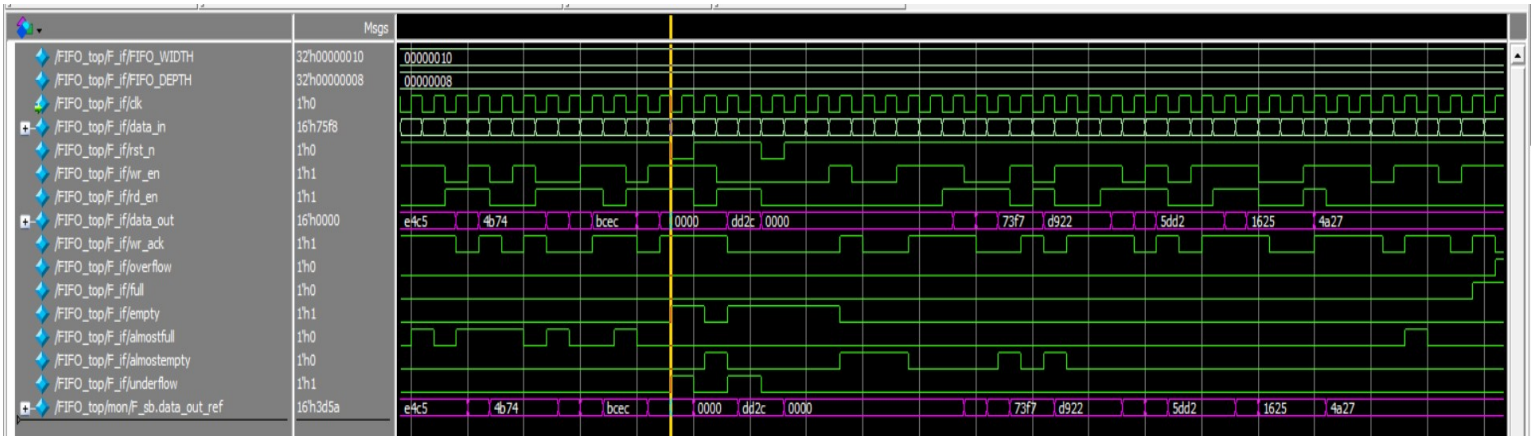
task assert_reset;
    //assert reset
     F_if.rst_n = 0;
     F_if.wr_en   = 0;
     F_if.rd_en   = 0;
     F_if.data_in = 0;
    #20;
    //desert reset
    F_if.rst_n = 1;
endtask

endmodule
```

TOP

```
codes > FIFO_top.sv > FIFO_top
1   module FIFO_top();
2   bit clk;
3   initial begin
4       clk = 0;
5       forever
6        #1 clk =~clk;
7   end
8           FIFO_interface F_if(clk);
9           FIFO dut(F_if);
10          FIFO_tb tb(F_if);
11          FIFO_monitor mon(F_if);
12  endmodule
```

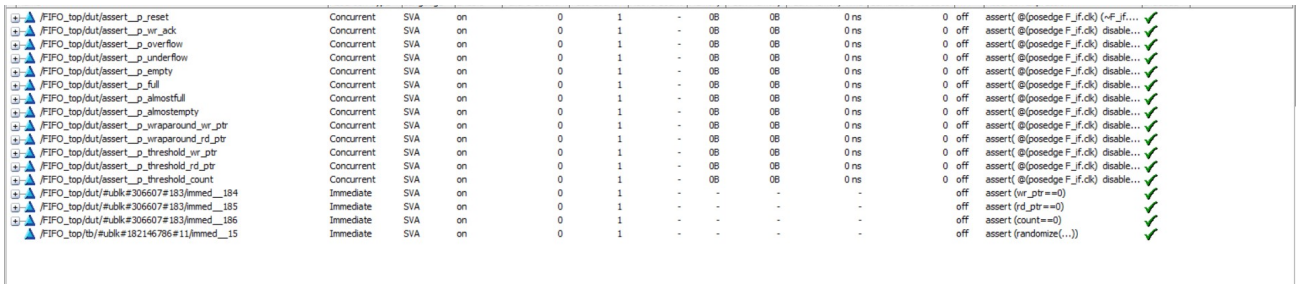
Snippet from Waveform



Snippet from transcript

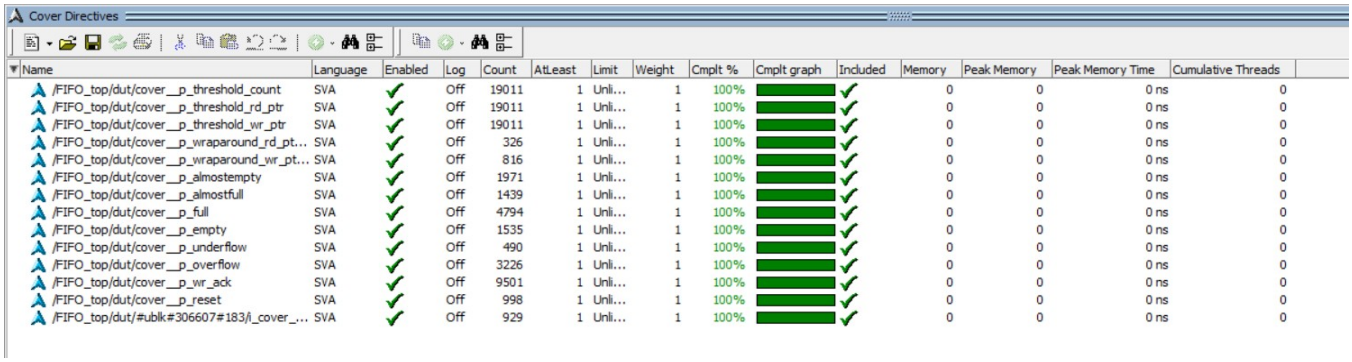
```
# ========================================
# Simulation Finished
# Correct Count = 20000
# Error Count   = 0
# ========================================
```

Assertions



/FIFO_top/dut/assert__p_reset	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge F_if.clk) (~F_if....
/FIFO_top/dut/assert__p_wr_ack	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge F_if.clk) disable...
/FIFO_top/dut/assert__p_overflow	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge F_if.clk) disable...
/FIFO_top/dut/assert__p_underflow	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge F_if.clk) disable...
/FIFO_top/dut/assert__p_empty	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge F_if.clk) disable...
/FIFO_top/dut/assert__p_full	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge F_if.clk) disable...
/FIFO_top/dut/assert__p_almostfull	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge F_if.clk) disable...
/FIFO_top/dut/assert__p_almostempty	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge F_if.clk) disable...
/FIFO_top/dut/assert__p_wraparound_wr_ptr	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge F_if.clk) disable...
/FIFO_top/dut/assert__p_wraparound_rd_ptr	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge F_if.clk) disable...
/FIFO_top/dut/assert__p_threshold_wr_ptr	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge F_if.clk) disable...
/FIFO_top/dut/assert__p_threshold_rd_ptr	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge F_if.clk) disable...
/FIFO_top/dut/assert__p_threshold_count	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge F_if.clk) disable...
/FIFO_top/dut/#ublk#306607#183/immed__184	Immediate	SVA	on	0	1	-	-	-	-		off	assert (wr_ptr==0)
/FIFO_top/dut/#ublk#306607#183/immed__185	Immediate	SVA	on	0	1	-	-	-	-		off	assert (rd_ptr==0)
/FIFO_top/dut/#ublk#306607#183/immed__186	Immediate	SVA	on	0	1	-	-	-	-		off	assert (count==0)
/FIFO_top/tb/#ublk#18214786#11/immed__15	Immediate	SVA	on	0	1	-	-	-	-		off	assert (randomize(...))

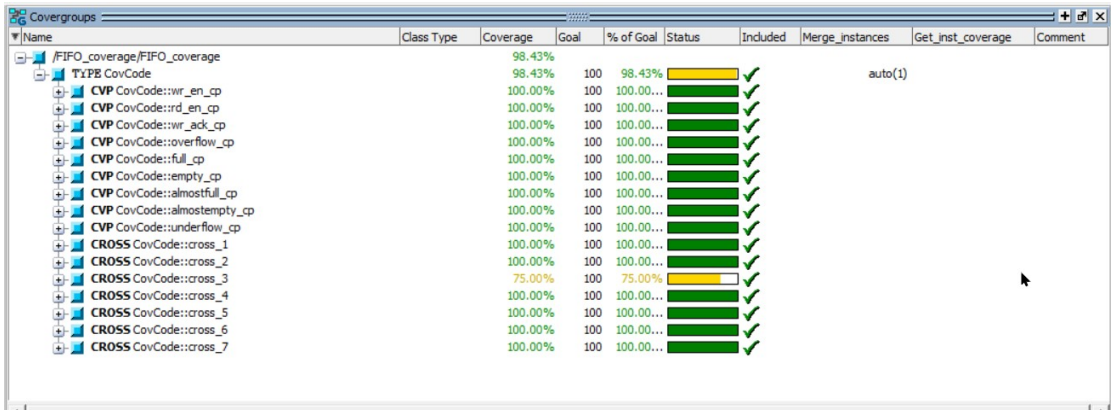
Assertions coverage



Cover Directives

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak Memory Time	Cumulative Threads
/FIFO_top/dut/cover__p_threshold_count	SVA	✓	Off	19011	1	Unli...	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/cover__p_threshold_rd_ptr	SVA	✓	Off	19011	1	Unli...	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/cover__p_threshold_wr_ptr	SVA	✓	Off	19011	1	Unli...	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/cover__p_wraparound_rd_pt...	SVA	✓	Off	326	1	Unli...	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/cover__p_wraparound_wr_pt...	SVA	✓	Off	816	1	Unli...	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/cover__p_almostempty	SVA	✓	Off	1971	1	Unli...	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/cover__p_almostfull	SVA	✓	Off	1439	1	Unli...	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/cover__p_full	SVA	✓	Off	4794	1	Unli...	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/cover__p_empty	SVA	✓	Off	1535	1	Unli...	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/cover__p_underflow	SVA	✓	Off	490	1	Unli...	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/cover__p_overflow	SVA	✓	Off	3226	1	Unli...	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/cover__p_wr_ack	SVA	✓	Off	9501	1	Unli...	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/cover__p_reset	SVA	✓	Off	998	1	Unli...	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/#ublk#306607#183/i_cover_...	SVA	✓	Off	929	1	Unli...	1	100%		✓	0	0	0 ns	0

Functional coverage



Covergroups

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/FIFO_coverage/FIFO_coverage		98.43%							
TYPE CovCode		98.43%	100	98.43%		✓		auto(1)	
CVP CovCode::wr_en_cp		100.00%	100	100.00...		✓			
CVP CovCode::rd_en_cp		100.00%	100	100.00...		✓			
CVP CovCode::wr_ack_cp		100.00%	100	100.00...		✓			
CVP CovCode::overflow_cp		100.00%	100	100.00...		✓			
CVP CovCode::full_cp		100.00%	100	100.00...		✓			
CVP CovCode::empty_cp		100.00%	100	100.00...		✓			
CVP CovCode::almostfull_cp		100.00%	100	100.00...		✓			
CVP CovCode::almostempty_cp		100.00%	100	100.00...		✓			
CVP CovCode::underflow_cp		100.00%	100	100.00...		✓			
CROSS CovCode::cross_1		100.00%	100	100.00...		✓			
CROSS CovCode::cross_2		100.00%	100	100.00...		✓			
CROSS CovCode::cross_3		75.00%	100	75.00%		✓			
CROSS CovCode::cross_4		100.00%	100	100.00...		✓			
CROSS CovCode::cross_5		100.00%	100	100.00...		✓			
CROSS CovCode::cross_6		100.00%	100	100.00...		✓			
CROSS CovCode::cross_7		100.00%	100	100.00...		✓			



CROSS CovCode::cross_3	75.00%	100	75.00%	✓
bin <wr_en_1,rd_en_0,full_1>	4067	1	100.00...	✓
bin <wr_en_0,rd_en_0,full_1>	973	1	100.00...	✓
bin <wr_en_1,rd_en_1,full_0>	4214	1	100.00...	✓
bin <wr_en_0,rd_en_1,full_0>	1786	1	100.00...	✓
bin <wr_en_1,rd_en_0,full_0>	5807	1	100.00...	✓
bin <wr_en_0,rd_en_0,full_0>	3153	1	100.00...	✓
bin <wr_en_1,rd_en_1,full_1>	0	1	0.00%	✓
bin <wr_en_0,rd_en_1,full_1>	0	1	0.00%	✓

Coverage Analysis Report

Total functional coverage reached **98.43%**. The two missing cross-coverage bins (<wr_en_1, rd_en_1, full_1> and <wr_en_0, rd_en_1, full_1>) are rare corner cases.

Since full is an internal state rather than a direct input, the statistical probability of hitting these exact conditions during standard constrained-random simulation is extremely low. Covering them requires a **directed sequence** to intentionally fill the FIFO first, which falls outside the scope of the current randomized testing.

Toggle coverage

```
================================================================================
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         86        86         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /FIFO_top/F_if --

                                          Node      1H->0L      0L->1H  "Coverage"
                                          ----------------------------------------
                                   almostempty           1           1      100.00
                                    almostfull           1           1      100.00
                                           clk           1           1      100.00
                                  data_in[15-0]          1           1      100.00
                                 data_out[15-0]          1           1      100.00
                                         empty           1           1      100.00
                                          full           1           1      100.00
                                      overflow           1           1      100.00
                                         rd_en           1           1      100.00
                                         rst_n           1           1      100.00
                                     underflow           1           1      100.00
                                        wr_ack           1           1      100.00
                                         wr_en           1           1      100.00

Total Node Count     =         43
Toggled Node Count   =         43
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (86 of 86 bins)
```

Statement coverage

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      22        22         0   100.00%
```

Branch coverage

```
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        25        25         0   100.00%
```

Do file

codes > run.do

```
quit -sim
vdel -all -lib work
vlib work
vlog -sv shared_pkg.sv
vlog -sv FIFO_transaction.sv
vlog -sv FIFO_coverage.sv
vlog -sv FIFO_scoreboard.sv
vlog -sv FIFO_interface.sv
vlog -sv +define+SIM FIFO.sv
vlog -sv FIFO_monitor.sv
vlog -sv FIFO_tb.sv
vlog -sv FIFO_top.sv
vsim -voptargs=+acc FIFO_top
run -all
```